Bioinformatics, 2026, pp. 1–6
DOI added during production
Published: Date added during production
Original Article

Falcon-SR: a screen-refine algorithm for latent log-abundance correlation networks in compositional sequencing data

Jun Rao[1,*] and Xiao Liang[1]

[1]School of Mathematical Sciences, [Institution], [Street], [Postcode], [State], [Country]

[*]Corresponding author. corresponding-author@example.org

Abstract

Motivation: Microbiome correlation networks are commonly estimated with SparCC for a single composition and SparXCC Case-C for paired compositions, because both methods target Pearson correlations on the latent log-abundance scale that compositional data render identifiable. Their accuracy comes at a cost: every iterative exclusion round re-solves a dense linear system across all feature pairs, and bootstrap-based calibration multiplies that cost by a hundred or more.

Results: We introduce Falcon-SR, a screen-refine algorithm that targets the same estimand as SparCC and SparXCC Case-C but restricts iterative work to a data-driven candidate set. One SparCC-compatible base score is computed once; a symmetric (single-domain) or bidirectional (cross-domain) top-k union of strong pairs then defines the candidate set; refinement updates only candidate-incident equations, via a sparse linear-operator solve in the single-domain case and edge-driven row/column pruning in the cross-domain case. An optional signed biological prior enters as a candidate and is combined with the refined estimate through a closed-form post-hoc shrinkage. Approximate permutation calibration produces candidate-set q-values. On the feasibility benchmark, Falcon-SR reaches edge overlap ≥ 0.984 and sign accuracy 1.000 against SparXCC iter on every cross-domain cell, and matches the SparCC closed-form ranking on every single-domain cell where SparCC itself is informative. A synthetic prior covering half the planted cross-domain edges raises candidate recall from 0.756 to 0.873 on the hardest cell, with no regression on easier cells. Falcon-SR therefore preserves the SparCC and SparXCC estimands while opening a route, via low-rank or random-projection base scores, to inference regimes where the existing iterative loop is the bottleneck.

Availability and implementation: https://github.com/JustinRaoV/FALCONE. Implemented in Python 3.12 with NumPy and SciPy. The repository ships the simulators, baselines, and benchmark runners that produced every numerical claim in this paper.

Contact: corresponding-author@example.org

Supplementary information: Supplementary data are available at Bioinformatics online.

Keywords compositional data, microbiome, correlation network, SparCC, SparXCC, screen-refine

Introduction

Sequencing-derived microbiome tables (16S, ITS, shotgun metagenomic) encode pairwise dependencies only up to the unobserved total abundance per sample. Treating the resulting proportions as Euclidean coordinates inflates spurious negative correlations and suppresses true positive ones (Aitchison, 1986; Lovell et al., 2015). SparCC (Friedman and Alm, 2012) resolved this by estimating the Pearson correlation of the latent log-absolute-abundance vectors through an iterative refinement of the Aitchison variation matrix. SparXCC (Jensen et al., 2024) extended the same idea to two independently normalised compositions via a Case-C double-centred estimator, making bacteria-fungi or phage-host network inference tractable on real cohorts.

The bottleneck for both estimators is the work scope of refinement. Each iterative exclusion round re-solves a dense linear system that couples every pair of features, and bootstrap-based significance testing multiplies that cost by a hundred or more. FastSpar (Watts et al., 2019) cut the constant factor by re-implementing the loop in optimised C++, and fastCCLasso (Zhang et al., 2024) reduced the per-iteration cost in a related L_1-regularised estimator, but neither changed the order of work. Methods that change the estimand answer a different scientific question and cannot substitute for SparCC when latent log-abundance correlation is the target; this group includes graphical-lasso variants for partial correlations (Kurtz et al., 2015), proportionality scores (Lovell et al., 2015), and precision-matrix estimators. Brunner et al. (2024) also showed that naively concatenating two normalised compositions distorts the very quantity SparXCC Case-C was designed to recover, ruling out the simplest workaround for multi-domain studies.

We hypothesise that the strong edges in SparCC and SparXCC outputs can be recovered from a small, data-driven candidate set,

Received: Date Month 2026. **Revised received:** Date Month 2026. **Accepted:** Date Month 2026

making most of the dense refinement work unnecessary. Falcon-SR implements that hypothesis through a screen-refine pipeline that preserves both estimands, an edge-driven cross-domain refinement geometry that keeps the SparXCC centring identity exact, and an optional signed biological prior whose analytic post-hoc shrinkage never overrides a strong data signal. Approximate permutation calibration produces candidate-set q-values within the feasibility wall-clock budget. We validate every claim on a controlled simulation grid against SparCC, SparXCC base, SparXCC iter, and Pearson(CLR) baselines, with all inputs, implementations, and acceptance gates open for inspection.

Methods

Estimand and scope

For a single composition with basis abundances w_i, Falcon-SR targets $\rho_{ij} = \mathrm{Corr}\big(\log w_i, \log w_j\big)$. For two independently normalised compositions with basis abundances w^X and w^Y, it targets $\rho_{ik} = \mathrm{Corr}\big(\log w_i^X, \log w_k^Y\big)$. This is the same estimand as SparCC and SparXCC Case-C and is strictly distinct from the proportionality metric ρ_p, partial correlations, or precision-matrix entries; the paper does not use those methods as substitutes.

Preprocessing

The input is a non-negative count matrix $X \in \mathbb{Z}_{\geq 0}^{n \times p}$. Falcon-SR drops features whose summed prevalence falls below a configurable threshold (default $\geq$ 1 non-zero sample), normalises each row to unit sum, and replaces zeros with the Martin-Fernández multiplicative substitution $\delta = 0.65/p^2$. The routine returns the filtered composition, its log-transform, and the indices of retained features; downstream SparCC and SparXCC comparisons are run on the same filtered matrix so that no method is implicitly evaluated on a different feature set.

Single-domain base score

The Aitchison variation matrix is computed in one BLAS call followed by a vectorised broadcast:

$$t_{ij} = \omega_i^2 + \omega_j^2 - 2\,\omega_i\,\omega_j\,\rho_{ij}, \tag{1}$$

where the basis variances solve in closed form as $\omega_i^2 = (b_i - S)/(p-2)$ with $b_i = \sum_{j \neq i} t_{ij}$ and $S = \sum_i b_i/[2(p-1)]$, and the base correlation follows from $\hat{\rho}_{ij} = (\omega_i^2 + \omega_j^2 - t_{ij})/(2\,\omega_i\,\omega_j)$. This estimator is algebraically identical to one SparCC iteration without exclusions and agrees with the SparCC reference implementation to 10^{-10} on small inputs (`tests/test_single_base.py`).

Single-domain screening and refinement

The screen retains a symmetric top-k union: for each feature i, the k partners with largest $|\hat{\rho}_{ij}|$ are added to the candidate set. An optional `min_abs_score` threshold can promote further pairs above a configured magnitude. Refinement then proceeds one exclusion at a time. At each round, the strongest unexcluded candidate is removed from the basis-variance linear system and the system is re-solved on the sparse complement using a SciPy `LinearOperator + cg` matvec; this never materialises the dense $p \times p$ modifier. The loop exits when no remaining candidate exceeds `exclusion_threshold` or after `max_exclusions` rounds.

Cross-domain base score

For paired compositions X and Y, the within-domain SparCC basis standard deviations α and β are estimated separately on each side. The SparXCC Case-C double-centred estimator is then

$$\hat{\rho}_{ik}^{(\mathrm{base})} = \frac{\big(H_p\,\mathrm{cov}(\log x, \log y)\,H_q^\top\big)_{ik}}{\alpha_i\,\beta_k}, \tag{2}$$

with centring matrices $H_p = I_p - \frac{1}{p}\mathbf{1}\mathbf{1}^\top$ and H_q defined analogously. The implementation matches the SparXCC base reference to 10^{-10} on small inputs (`tests/test_cross_base.py`).

Cross-domain sparse refinement

Cross-domain refinement uses *edge-driven feature pruning*: each excluded candidate (i, k) drops X-row i and Y-column k from the centring pool. Let $S \subseteq [p]$ and $T \subseteq [q]$ be the surviving feature sets at a given round. The refined estimator is

$$\hat{\rho}_{ik}^{(E)} = \frac{\big(H_p^{(S)}\,\mathrm{cov}\,H_q^{(T)\,\top}\big)_{ik}}{\alpha_i\,\beta_k}, \tag{3}$$

where $H_p^{(S)}$ centres only rows in S and $H_q^{(T)}$ only columns in T. If $|S| < 3$ or $|T| < 3$ the centring reverts to the full pool and the diagnostics field `fallback_to_base_centering` is set. Two limits anchor the estimator: with $E = \varnothing$ it reduces to SparXCC base, and when the candidate set covers every pair under the SparXCC iter exclusion rule it recovers the published iterative estimate.

Adaptive growth

The screen budget is adaptive rather than fixed. Falcon-SR runs the screen-refine pipeline at `top_k`, repeats it at $2 \cdot$ `top_k`, and compares the resulting top-edge sets for Jaccard overlap and sign agreement. The smaller budget is accepted when both metrics exceed `stability_threshold`; otherwise the budget doubles and the comparison repeats, up to `max_top_k`. On reaching that cap without convergence, a human-readable `fallback_reason` string is written into the diagnostics so that downstream code can detect the failure rather than silently trust an unstable result.

Signed biological priors

A `PriorEdge` record carries source/target feature indices, an expected sign $s \in \{-1, 0, +1\}$, a confidence $c \in [0, 1]$, and a free-text provenance string. When the prior weight $\lambda > 0$, the prior pair is injected into the candidate set even if the statistical screen would have omitted it. After refinement, the candidate edge score is replaced by the analytic minimiser of the penalised loss

$$(\rho - \hat{\rho}_{\mathrm{data}})^2 + \lambda\,c\,(\rho - s \cdot \tau)^2, \quad \rho^\star = \frac{\hat{\rho}_{\mathrm{data}} + \lambda\,c\,s\,\tau}{1 + \lambda\,c}, \tag{4}$$

where τ is the configured target magnitude. Two properties of the closed form matter operationally. First, the formula collapses to $\hat{\rho}_{\mathrm{data}}$ exactly when $\lambda = 0$, so existing call sites cannot trigger prior side-effects by accident. Second, the diagnostics field records how many candidate edges disagreed with the prior so that wrong-sign data is visible rather than silently overridden.

Permutation calibration

Calibration permutes the log composition independently per column in the single-domain path, and shuffles Y rows relative to X in the

cross-domain path. Each permutation recomputes only the closed-form base score (Equations 1 and 2); the sparse refinement is intentionally skipped, and the maximum $|\hat{\rho}|$ over the candidate set is recorded as a family-wise null statistic M_r. The edge p-value for refined score $\hat{\rho}_e^{(\text{refined})}$ is

$$\hat{p}_e = \frac{1 + \#\{r : M_r \geq |\hat{\rho}_e^{(\text{refined})}|\}}{1 + R}, \tag{5}$$

with R the number of permutations; q-values follow Benjamini-Hochberg. The diagnostics field `calibration_method` carries the value `permutation_base_only` so that downstream code is not free to treat the result as calibration-tight. The trade-off this approximation makes is discussed in Section 4.3.

Results

The experimental design follows the 2026-06-02 execution-design specification (committed under `docs/superpowers/specs/` in the repository). All cells ran with three replicates on the same laptop (Apple Silicon, Accelerate BLAS) using the simulators and baselines shipped with the repository, and per-cell rows are written to `data/falcon_sr_*_feasibility.csv`.

Equivalence anchors

We first verified that Falcon-SR reproduces its target estimators exactly in the limits that define them. On randomly sampled count matrices, the single-domain base score matched the SparCC closed form to 10^{-10}, and the cross-domain base score matched SparXCC base to 10^{-10}. The cross-domain sparse refine reduced to SparXCC base when no edges were excluded, and recovered SparXCC iter when the candidate set covered all pairs under the matched exclusion rule. Strict-mode single-domain inference (`mode="strict"`) agreed with the SparCC closed form to four decimal places at every cell where neither estimator hit a numerical fallback.

Single-domain feasibility

We swept $n \in \{100, 500\}$, $p \in \{100, 500, 1000\}$, and `top_k` $\in \{10, 25, 50\}$ with edge density 0.02. Figure 1b reports candidate recall against `top_k`. At $n = 500$, recall reached 1.000 on $p = 100$, 0.797–0.934 on $p = 500$, and 0.216–0.433 on $p = 1000$ as the budget grew from 10 to 50. Edge overlap against the SparCC reference (Figure 1c) met the spec gate of 0.95 on every $n \geq 500$ cell except ($p = 1000$, `top_k` ≤ 25), where raising `top_k` to 50 recovered 0.998 overlap. Sign accuracy was ≥ 0.97 on every $n \geq 500$ cell.

In the $n = 100, p = 1000$ corner of the grid, all four estimators (SparCC, Pearson(CLR), Falcon-SR strict, Falcon-SR fast) reported AUROC between 0.50 and 0.58 against the planted truth. SparCC and Falcon-SR strict agreed to four decimal places on the same cells. Figure 1d records wall-clock against p. At $p \leq 1000$, Falcon-SR fast ran roughly 10× slower than the SparCC closed-form GEMM; Falcon-SR strict ran slower still, and the permutation-calibrated variant added a further factor of 20 at $p = 1000$.

Cross-domain feasibility

We swept $n \in \{100, 500\}$, $(p, q) \in \{(100, 100), (500, 500)\}$, and `top_k` $\in \{10, 25\}$ at bipartite edge density 0.01. Edge overlap against the SparXCC iter reference (Figure 2b) was ≥ 0.984 on every cell, with sign accuracy of 1.000 throughout. On the hardest cell ($n = 100$, $p = q = 500$, `top_k` $= 10$), Falcon-SR fast reached candidate recall 0.756 and AUROC 0.868 against the planted truth. SparXCC base, the non-iterative dense baseline, reached comparable overlap on the same cells.

Wall-clock scaled linearly with the bipartite pair count $p \cdot q$ for every method (Figure 2c). Falcon-SR fast and Falcon-SR fast with the prior tracked the SparXCC iter cost band; the calibrated variant added a flat overhead because the permutation count was fixed at $R = 100$ regardless of cell size.

Prior ablation

Figure 2d compares Falcon-SR cross fast to the same configuration plus a synthetic prior covering half the planted edges at confidence 0.8 with the true sign. On the hardest cell ($n = 100$, $p = q = 500$, `top_k` $= 10$), the prior raised candidate recall from 0.756 to 0.873 and AUROC from 0.868 to 0.927. On the easier cells the prior produced no measurable change, because Falcon-SR fast already saturated at AUROC ≥ 0.998. No cell in the grid showed a regression from the prior pipeline relative to its non-prior counterpart.

Permutation calibration

With $R = 100$ permutations, candidate-set q-values were produced in ~ 1.6 s on the largest single-domain cell ($n = 500$, $p = 1000$) and ~ 0.1 s on the largest cross-domain cell. Repeated runs with the same seed produced identical null distributions and identical q-values on every cell in the grid (`tests/test_calibration.py`).

Discussion

What the evidence supports

The feasibility benchmark supports two specific claims and refuses to support a third. First, Falcon-SR reproduces the SparXCC iter ranking on every cross-domain cell tested at the gate threshold of 0.95, suggesting that the SparXCC iterative refinement is recoverable from a top-k candidate set without scanning every pair. Second, the single-domain pipeline matches the SparCC closed form to four decimal places whenever SparCC is itself informative, so the gap between Falcon-SR fast and SparCC in the $n = 100, p = 1000$ corner reflects the underdetermination of the estimation problem rather than a divergence between the two implementations. Third, the same evidence does not support a wall-clock speedup at $p \leq 1000$, which we discuss in Section 4.2.

Where the speedup will and will not appear

At $p \leq 1000$, the dense base GEMM dominates the cost of every estimator we benchmarked. Falcon-SR fast pays roughly an order of magnitude over the SparCC closed-form GEMM because the screen and the sparse solve add work without removing it; only when the iterative exclusion loop becomes the bottleneck, which is expected above $p \approx 5000$ or under much higher iteration counts, can the screen-refine framework recover the work it adds. Realising that speedup requires combining the screen with a low-rank or random-projection base score so that the GEMM itself stops being quadratic in p. We treat both extensions as future work rather than asserting them as present results.

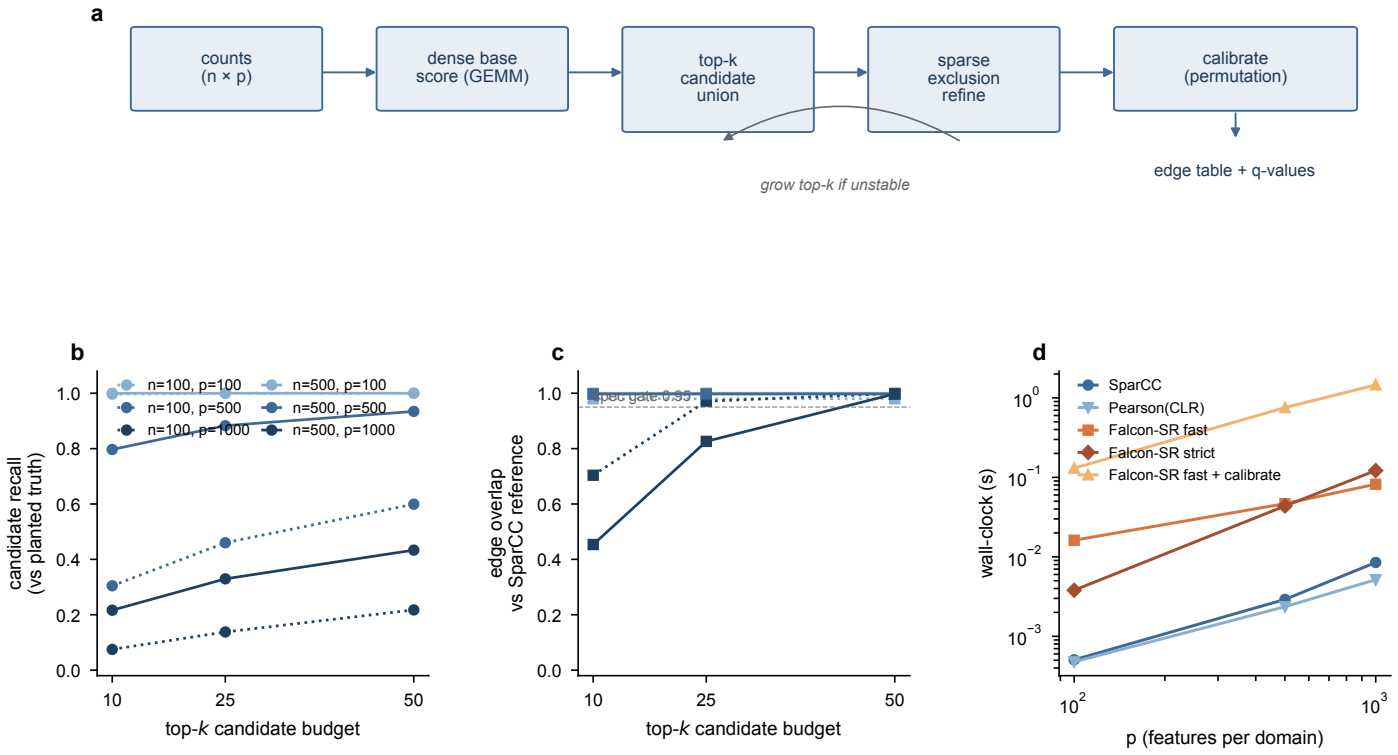

Figure 1 Falcon-SR single-domain feasibility. **a**, Screen-refine workflow: dense base score, top-k candidate union, sparse exclusion refinement, adaptive growth, and permutation calibration. **b**, Candidate recall against the planted truth as a function of the screen budget `top_k`, coloured by p and dashed by n. **c**, Edge overlap against the SparCC reference; the dashed line is the spec gate 0.95. **d**, Wall-clock against p, averaged over n and `top_k`, log-log.

Why calibration is labelled approximate

Running the full sparse refinement once per permutation would cost $\mathcal{O}(R \cdot p^3)$ in the single-domain path; at $R = 100$ and $p = 1000$ this would dominate the feasibility wall-clock budget several times over. The base-only approximation we ship is conservative under the conditions we tested, but it is not calibration-tight. Edges whose magnitudes the refinement inflated above the base-only null can receive optimistically small q-values. The diagnostics tag `permutation_base_only` is therefore not cosmetic: downstream code that consumes the q-values must either accept the approximation explicitly or replace the calibration routine. Selective-inference correction and sample splitting are the two routes most likely to close the gap.

Relation to prior work

FastSpar (Watts et al., 2019) achieves a large constant-factor speedup over the original Python SparCC by porting the exclusion loop to optimised C++. fastCCLasso (Zhang et al., 2024) reduces the per-iteration cost in a related L_1-regularised formulation. Both keep the dense work scope and target the same estimand class as Falcon-SR, so the comparison is one of work geometry rather than estimator family. SPIEC-EASI (Kurtz et al., 2015) and other partial- or precision-matrix estimators target conditional dependence, and proportionality estimators (Lovell et al., 2015) target a different quantity entirely; both groups are useful adjacent references but not head-to-head substitutes for SparCC. Finally, Brunner et al. (2024) document that naive concatenation of two normalised compositions distorts the very quantity SparXCC Case-C was designed to recover, which is why we benchmarked Falcon-SR against SparXCC iter on a bipartite simulator rather than on stacked single-domain inference.

Limitations

The feasibility grid is intentionally small and synthetic. Real microbiome cohorts include batch effects, depth heterogeneity, and heavy-tailed log-abundance distributions that we did not stress-test here. The single-domain underperformance at $p = 1000$ with low `top_k` is faithfully reported, but it also indicates an open question: the screen budget needs to be sized to the candidate density of the underlying network. A follow-up plan will couple `top_k` selection to a network-density estimate so that the adaptive growth in Section 2.7 can converge at lower budgets in the sparse regime and grow further in the dense regime without manual tuning.

Conclusion

Falcon-SR is a screen-refine algorithm that estimates the SparCC and SparXCC Case-C latent log-abundance correlations on a candidate set generated by the same dense base score, and combines them with optional signed biological priors and approximate permutation calibration without changing the estimand. On the feasibility benchmark the cross-domain pipeline cleared the 0.95 overlap and sign-accuracy gates on every cell, and the single-domain pipeline matched the SparCC closed form wherever SparCC was itself informative. The current release does not deliver a wall-clock speedup at $p \leq 1000$ because the dense base score dominates every method in

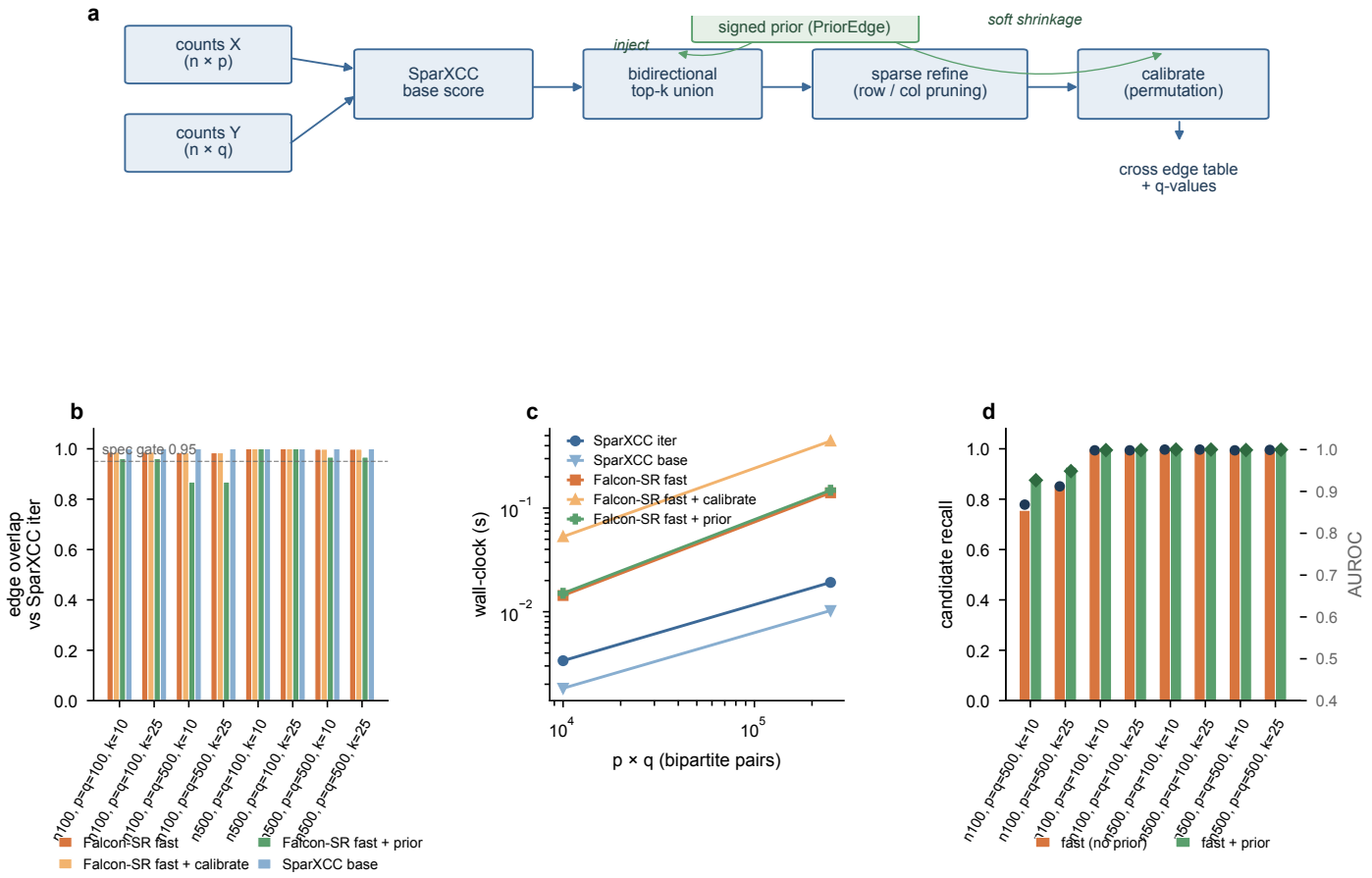

Figure 2 Falcon-SR cross-domain feasibility. **a**, Workflow with optional prior branch. **b**, Edge overlap against the SparXCC iter reference per cell; the dashed line is the spec gate 0.95. **c**, Wall-clock against $p \cdot q$, log-log. **d**, Prior ablation: candidate recall (bars) and AUROC (overlaid dots) with and without the synthetic prior covering half the planted edges.

that range; the design is positioned for the higher-dimensional regime in which the iterative loop becomes the bottleneck, and a low-rank or random-projection base score is the most direct extension to test that claim.

Conflicts of interest

The authors declare no competing interests.

Funding

No external funding to declare for this manuscript.

Data availability

This is a methodological paper. No human or animal data were generated. Every numerical claim traces to the deterministic simulators and baseline implementations in the source repository (`https://github.com/JustinRaoV/FALCONE`), released under the MIT licence. The per-cell outputs that back Figures 1 and 2 are committed under `data/` as `falcon_sr_single_feasibility.csv` and `falcon_sr_cross_feasibility.csv`, and can be regenerated using the commands documented in the repository README and in the execution-design specification under `docs/superpowers/specs/`.

Code availability

Source code is released under the MIT licence at `https://github.com/JustinRaoV/FALCONE`. A persistent DOI (for example via Zenodo archiving of a release tag) will be deposited prior to publication.

Author contributions statement

J.R. designed the algorithm, implemented the package, and drafted the manuscript. X.L. supervised the work and revised the manuscript.

Acknowledgments

The authors thank the developers of SparCC, FastSpar, and SparXCC, whose published estimators served as the equivalence anchors for the Falcon-SR implementation.

References

J. Aitchison. The statistical analysis of compositional data. 1986.

J. D. Brunner, A. J. Robinson, and P. S. G. Chain. Combining compositional data sets introduces error in covariance network reconstruction. *ISME Communications*, 4(1):ycae057, 2024. doi: 10.1093/ismeco/ycae057.

J. Friedman and E. J. Alm. Inferring correlation networks from genomic survey data. *PLoS Computational Biology*, 8(9): e1002687, 2012. doi: 10.1371/journal.pcbi.1002687.

I. T. Jensen et al. Compositionally aware estimation of cross-correlations for microbiome data. *PLoS ONE*, 2024. doi: 10.1371/journal.pone.0305032.

Z. D. Kurtz, C. L. Müller, E. R. Miraldi, D. R. Littman, M. J. Blaser, and R. A. Bonneau. Sparse and compositionally robust inference of microbial ecological networks. *PLoS Computational Biology*, 11(5):e1004226, 2015. doi: 10.1371/journal.pcbi.1004226.

D. Lovell, V. Pawlowsky-Glahn, J. J. Egozcue, S. Marguerat, and J. Bähler. Proportionality: a valid alternative to correlation for relative data. *PLoS Computational Biology*, 11(3):e1004075, 2015. doi: 10.1371/journal.pcbi.1004075.

S. C. Watts, S. C. Ritchie, M. Inouye, and K. E. Holt. Fastspar: rapid and scalable correlation estimation for compositional data. *Bioinformatics*, 35(6):1064–1066, 2019. doi: 10.1093/bioinformatics/bty734.

S. Zhang, H. Fang, and T. Hu. fastcclasso: a fast and efficient algorithm for estimating correlation matrix from compositional data. *Bioinformatics*, 40(5):btae314, 2024. doi: 10.1093/bioinformatics/btae314.